EGER 1774
ESZTERHÁZY KÁROLY KATOLIKUS EGYETEM
Matematikai és Informatikai Intézet

Informatikai eszközök nyilvántartására, eszközkiadás kezelésére tervezett alkalmazás készítése

Készítette
Csik Patrik
Programtervező informatikus

Témavezető
Dr. Király Roland
Egyetemi docens

Eger, 2026

Tartalomjegyzék

1. Bevezetés **4**
1.1. A téma aktualitása és célja 4
1.2. A használt technológiák bemutatása 6
1.2.1. Microsoft .NET 8 Web API 7
1.2.2. Microsoft SQL Server 7
1.2.3. React 8
1.2.4. Fejlesztői környezet és eszközök 9

2. A rendszer célja és funkcionális követelményei **11**
2.1. A rendszer célja 11
2.2. Funkcionális követelmények 12
2.2.1. Bejelentkezés és hitelesítés 12
2.2.2. Felhasználói szerepkör 12
2.2.3. Vezetői szerepkör 14
2.2.4. Adminisztrátori szerepkör 15
2.2.5. Nem funkcionális követelmények 15
2.2.6. Hasonló rendszerek bemutatása 16
2.2.7. Összegzés 16

3. Tervezés **17**
3.1. Rendszer architektúra 17
3.2. Adatbázis tervezés 18
3.3. Backend tervezése 19
3.4. Frontend tervezése 19
3.5. Autentikáció és jogosultságkezelés 20
3.6. JSON Web Token működése 22

4. Fejlesztés és megvalósítás **23**
4.1. Fejlesztési környezet 23
4.2. Backend fejlesztés 23
4.3. Frontend fejlesztés 26
4.4. Integráció 28

4.5. Biztonság és naplózás 28

5. Tesztelés **29**

5.1. Tesztelési módszertan 29

5.2. Tesztesetek és eredmények 30

5.3. API tesztelés (Postman) 31

5.4. Összegzés 31

6. Képernyőképek és felhasználói felületek bemutatása **32**

7. További fejlesztési lehetőségek **35**

8. Összefoglalás és köszönetnyilvánítás **36**

9. Forráskód és demó alkalmazás elérhetősége **37**

1. fejezet

Bevezetés

1.1. A téma aktualitása és célja

Kis és középvállalatoknál végzett szakmai munkáim során gyakran azt tapasztaltam, hogy a cégnél használt IT eszközök (számítógépek, laptopok, mobiltelefonok stb.) nem megfelelő nyilvántartása megnehezíti a mindennapi feladatok elvégzését az IT osztályon belül és esetlegesen anyagi károk is jelenthet a munkaadó számára.

A digitalizáció a jelenlegi ipari környezeten belül alapkövetelménynek tekinthető, ennek következtében az intézményekben használt korábban említett eszközök darabszáma is drasztikusan megnőtt. Az igények folyamatos változása és a használatban lévő eszközök darabszámának növekedése miatt ezeknek az IT eszközöknek a nyilvántartása létfontosságú a vállalati életben.

Tapasztalataim alapján jellemző a különböző táblázatkezelő alkalmazások használata (például Microsoft Excel), melyeknek hátránya, hogy az adott felhasználó minden adatot szabadon tud szerkeszteni, a meglévő adatstruktúrát módosíthatja. Egy közepes méretű cég esetén ez akár több száz különböző eszközt is jelenthet, amiknek a mozgását, státuszát és felelősét nyilvántartani egy táblázatkezelőben nem fenntartható hosszú távon. Az ilyen megoldások alkalmazásakor gyakori az adatvesztés, idővel a struktúra teljes összeomlása és az „Egyéb” mezők rendszeres megjelenése az „adatbázisban”.

Szakdolgozatom célja, egy olyan webalkalmazás fejlesztése ami a korábban említett problémákra biztosít egy lehetséges megoldást. A program egy olyan környezetet szolgáltat a felhasználóknak és az IT munkatársaknak amin keresztül a felhasználók egyszerűen tudnak új eszközöket igényelni, megtekinthetik a jelenlegi eszközeiket és állapotukat, a vezetők jóváhagyási folyamaton keresztül tudják felülvizsgálni az alkalmazottak igényeit, az adminisztrátorok pedig teljes körű rálátást kapnak a telephelyen lévő összes eszközre.

Az alkalmazásom által az eszközök teljes életciklusa digitálisan nyomon követhetővé válik, ezáltal csökken az adminisztrációs teher és minimalizálható az emberi hibákból

eredő adatvesztés.

A tervezés során az alábbi célokat határoztam meg:

- Eszközök (számítógépek, telefonok, SIM kártyát) kezelése
- Igénylési folyamat megvalósítása
- Jóváhagyási folyamat megvalósítása
- Igénylés feldolgozásának megvalósítása
- Átadási / Átvételi dokumentum digitalizációja
- Használatban lévő eszközök életciklusának követhetősége
- Felhasználó és jogosultság kezelés

Célom, hogy az alkalmazásom bevezetésével növelem az átláthatóságot és használatával csökkenthetem az üzemeltetésre, adminisztrációra fordított kapacitás mennyiségét. A fejlesztés során személyes célom, hogy elmélyítsem a tudásomat a piacképes technológiák használatában és az alkalmazásfejlesztés vállalati szintű gyakorlatában.

A szakdolgozatom további fejezetei a rendszer tervezését, megvalósítását és értékelését mutatják be.

A következő fejezetben bemutatom a rendszer tervezését amely során ismertetésre kerül a választott architektúra, az adatbázis felépítése valamint a backend és frontend komponensek kialakítása. A fejezet célja a rendszer struktúrájának és működési alapelveinek bemutatása.

A harmadik fejezet a fejlesztés és megvalósítás részleteit tartalmazza. Itt kerül bemutatásra a backend és frontend komponensek működése, az alkalmazott technológiák használata, valamint a rendszer különböző funkcióinak implementációja.

A negyedik fejezet a tesztelési folyamatot ismerteti, amely során a rendszer működésének ellenőrzése történik különböző tesztesetek segítségével. A fejezet célja annak igazolása, hogy az alkalmazás megfelel a korábban meghatározott követelményeknek.

Az ötödik fejezet a rendszer értékelését tartalmazza, amelyben a megvalósított megoldás előnyei és esetleges korlátai kerülnek bemutatásra.

A hatodik fejezet a továbbfejlesztési lehetőségeket ismerteti, bemutatva azokat az irányokat, amelyek mentén a rendszer a jövőben tovább bővíthető.

Végül az utolsó fejezet összefoglalja a dolgozat során elvégzett munkát, és értékeli a kitűzött célok megvalósulását.

1.2. A használt technológiák bemutatása

A munkám során arra törekedtem, hogy olyan technológiai eszközöket alkalmazzak, melyek illeszkednek a jelenlegi iparági trendekhez és megfelelnek a vállalati fejlesztési eszközök elvárásainak.

A rendszer tervezése során a gyakran alkalmazott háromrétegű architektúrát választottam, amely az alkalmazást megjelenítési, üzleti logikai és adatkezelési rétegekre bontja.

Az üzleti logika megvalósítására több elérhető technológia közül a Microsoft .Net 8 Web API-t választottam amely magas teljesítményű környezetet biztosít. Azért esett a választásom a Microsoft által biztosított megoldásra, mert a jelenlegi üzleti környezetben, ahová az alkalmazást készítem a .Net egy központilag támogatott megoldás és futtatókörnyezetként Windows szervereket alkalmazunk ami lehetővé teszi az egyszerű integrációt.

Lehetőségem lett volna PHP Laravel keretrendszer alkalmazása, mivel az egyetemi tanulmányaim során megszereztem a szükséges tudást. A meglévő tudásomtól független a .Net megoldást választottam, hiszen népszerűsége jelentősebb a fejlesztők körében és szerettem volna egy piacképes tudást felépíteni. [4] Emellett a .Net keretrendszer teljesítménye optimálisabb közepes és nagy méretű alkalmazások vállalati környezetben való megvalósítására. Mindemellett előnye a skálázhatóság így az alkalmazás könnyedén fel tudtam készíteni a növekvő terhelésre.

A megjelenítési réteg megvalósítására React könyvtárat alkalmaztam. A React egy komponens–alapú JavaScript könyvtár, amelyet dinamikus és gyors felhasználói felületek építésére használnak, különösen egyoldalas alkalmazások (SPA-k) esetén. [5] Hasonló környezetet biztosít az Angular keretrendszer is, de míg a React egy rugalmas könyvtárat biztosít, addig az Angular egy teljes értékű keretrendszer. Az Angular több beépített megoldást kínál, viszont komplexebb és tanulása több időt vesz igénybe. A React könyvtár egyszerűbb és nagyobb szabadságot ad a fejlesztés során, könnyebben testre szabhatóbb. Azért a React könyvtár használatát választottam, mivel egyszerűbb, rugalmasabb és jobban illeszkedik a projekt igényeihez.

Az adatok tárolására Microsoft SQL Szervert alkalmaztam, amely jól kezeli az adatok közötti kapcsolatokat és biztosítja az adatintegritást. Nagyvállalati környezetben gyakran alkalmazott az Oracle adatbázis is amely különösen kiemelkedő a skálázhatóság és a rendelkezésre állás terén, viszont a jelenlegi gazdasági helyzetben nagy hátrányt jelent számára a licenc ára. A Microsoft SQL szerver könnyen kezelhető, könnyebben integrálható a .NET alkalmazásokhoz és a Microsoft környezetbe. Teljesítmény terén mindkét adatbázis kiszolgáló megfelelő a tervezett alkalmazásom használatához, de a korábban említett környezet miatt az MS SQL szervert választottam.

Az általam választott technológiák egy jól skálázható, könnyen integrálható rend-

szert biztosítanak kis és középvállalati környezetben.

1.2.1. Microsoft .NET 8 Web API

A szerveroldali logika megvalósításához a .NET 8 Web API keretrendszert alkalmaztam.

A .NET egy ingyenes, platformfüggetlen, nyílt forráskódú fejlesztői platform, amely különféle alkalmazások felépítésére nyújt lehetőséget. A nagy teljesítményű futtatókörnyezetre támaszkodik, amelyet számos nagy léptékű alkalmazás használ éles környezetben. A .NET platform a termelékenység, a teljesítmény, a biztonság és a megbízhatóság biztosításához készült. [2]

Az adatbázissal való kommunikáció során az Entity Framework Core objektum-relációs leképező (ORM) keretrendszerrel valósítottam meg. Ez a rendszer lehetőség biztosít arra, hogy az adatbázis műveletek objektumorientált módon, C# nyelven kerüljenek megvalósításra, így elkerülve a kézzel írt SQL lekérdezéseket amely csökkenti a lehetséges hibák előfordulását.

Az Entity Framework (EF) Core a népszerű Entity Framework adatelérési technológia egyszerűsített, bővíthető, nyílt forráskódú és platformfüggetlen verziója.

Az EF Core objektum–relációs leképezőként (O/RM) szolgálhat, amely:

1. Lehetővé teszi, hogy a .NET-fejlesztők .NET-objektumokkal dolgozzanak egy adatbázissal.

2. Kiküszöböli az általában írandó adatelérési kódok többségének szükségességét.

Az EF Core alkalmazásával az adathozzáférés egy modell használatával történik. A modell entitásosztályokból és környezeti objektumból áll, amelyek az adatbázissal való munkamenetet jelölik. A környezeti objektum lehetővé teszi az adatok lekérdezését és mentését.[3]

A .NET 8 keretrendszerben megvalósított üzleti logika egy modern, nagy teljesítményű alkalmazás. Az adatkezelés során Entity Framework Core-t használtam, amely egyszerűsíti az adatbázis-műveletek megvalósítását és támogatja az objektumorientált megközelítést.

1.2.2. Microsoft SQL Server

Az alkalmazás adatainak tárolására a Microsoft SQL Server relációs adatbázis-kezelő kiszolgálót alkalmaztam. A választás során fontos szempont volt a megbízhatóság, a teljesítmény, valamint a .NET környezettel való egyszerű integráció.

A Microsoft SQL Server egy széles körben elterjedt, vállalati szintű adatbázis-kezelő rendszer, amely támogatja a strukturált adatok hatékony tárolását és kezelését. A

relációs modell alkalmazása lehetővé teszi az adatok közötti kapcsolatok egyértelmű definiálását, amely különösen fontos az eszközök, felhasználók és rendelések közötti összefüggések kezelésében.

Az adatbázis tervezése során normalizált adattáblákat hoztam létre, amelyek csökkentik az adatredundanciát és biztosítják az adatok konzisztenciáját. Az egyes entitások közötti kapcsolatokat idegen kulcsok segítségével valósítottam meg, amely biztosítja az adatintegritást.

Alternatív megoldásként szóba jöhettek volna más relációs adatbázis-kezelők, például a MySQL vagy a PostgreSQL. Ezek szintén megbízható és széles körben használt rendszerek, azonban a Microsoft SQL Server előnye a .NET környezettel való natív integráció, valamint a fejlesztői eszközök (pl. SQL Server Management Studio) által nyújtott kényelmes használat.

A Microsoft SQL Server a Microsoft által fejlesztett, zárt forráskódú relációs adatbázis-kezelő rendszer, amely a Structured Query Language (SQL) nyelvet használja. Adatbázis-szerverként működve elsődleges feladata az adatok tárolása és lekérdezése más szoftveralkalmazások kérésére. Ezek az alkalmazások futhatnak ugyanazon a számítógépen, vagy hálózaton keresztül – beleértve az internetet is – más számítógépeken.[7]

Összességében a Microsoft SQL Server használata stabil, jól skálázható és megbízható adatkezelést biztosít az alkalmazás számára, miközben hatékonyan támogatja a backend rendszer működését.

1.2.3. React

A frontend fejlesztése során a React JavaScript könyvtárat alkalmaztam, amely napjaink egyik legnépszerűbb megoldása dinamikus felhasználói felületek kialakítására. A React komponensalapú megközelítésével lehetővé teszi az alkalmazás felületének kisebb, újrafelhasználható egységekre bontását. Ez a struktúra megkönnyíti a fejlesztést, a karbantartást, valamint a kód újrafelhasználását.

A React egyik legfontosabb előnye a virtuális DOM (Document Object Model) használata, amely optimalizálja a felhasználói felület frissítését. Ennek köszönhetően csak azok az elemek kerülnek újrarenderelésre, amelyek ténylegesen megváltoztak, így javul az alkalmazás teljesítménye.

A fejlesztés során React hook-okat (például useState és useEffect) használtam az állapotkezelés és az életciklus események kezelésére. Ez lehetővé teszi a funkcionális komponensek használatát, amelyek egyszerűbb és átláthatóbb kódot eredményeznek.

Alternatív megoldásként szóba jöhetett volna az Angular keretrendszer, amely egy teljes körű frontend megoldást kínál. Az Angular számos beépített funkcióval rendelkezik, például dependency injection, routing és formkezelés, így egy komplexebb, strukturáltabb környezetet biztosít. Ugyanakkor a fejlesztés során több előre definiált

szabályhoz kell igazodni.

A React ezzel szemben nagyobb rugalmasságot biztosít, mivel nem egy teljes keretrendszer, hanem egy könyvtár, amelyhez a fejlesztő választhatja ki a szükséges kiegészítő eszközöket (például routing vagy állapotkezelés). Ez különösen előnyös kisebb vagy közepes méretű projektek esetén, ahol fontos a gyors fejlesztés és az egyszerűség.

Az Angular erőssége inkább nagyvállalati alkalmazások esetén jelentkezik, ahol a szigorúbb struktúra és a beépített megoldások előnyt jelentenek. Ezzel szemben a React könnyebb tanulhatósága és rugalmassága miatt szélesebb körben elterjedt, különösen modern webalkalmazások fejlesztése során.

A React könyvtár választása megfelelő döntésnek bizonyult, mivel jól illeszkedik a projekt követelményeihez, és lehetővé teszi egy dinamikus, jól karbantartható felhasználói felület kialakítását.

1.2.4. Fejlesztői környezet és eszközök

A fejlesztés során több, egymással jól együttműködő fejlesztői eszközt alkalmaztam, amelyek hozzájárultak a hatékony és strukturált munkavégzéshez. A backend fejlesztése Microsoft Visual Studio 2022 környezetben történt, míg a frontend fejlesztéshez Visual Studio Code szerkesztőt használtam.

A Microsoft Visual Studio 2022 egy teljes körű integrált fejlesztői környezet (IDE), amely kifejezetten jól támogatja a .NET alapú alkalmazások fejlesztését. Előnyei közé tartozik a fejlett hibakeresési lehetőség, az automatikus kódkiegészítés, valamint a beépített eszközök, amelyek megkönnyítik a projekt kezelését és a kód navigációját. A Visual Studio különösen előnyös nagyobb backend projektek esetén, ahol fontos a stabilitás és a komplex funkcionalitás támogatása.

A frontend fejlesztéshez a Visual Studio Code szerkesztőt alkalmaztam, amely egy könnyű, gyors és bővíthető eszköz. A Visual Studio Code támogatja a különböző programozási nyelveket és keretrendszereket, és számos kiegészítő segítségével testre szabható. A React fejlesztés során különösen hasznosak voltak az olyan bővítmények, amelyek támogatják a JavaScript és TypeScript fejlesztést, valamint az automatikus kódformázást.

Alternatív fejlesztői környezetként szóba jöhetett volna például a JetBrains által fejlesztett IntelliJ IDEA vagy WebStorm, amelyek szintén széles körű funkcionalitást biztosítanak. Ezek az eszközök különösen kiemelkedőek frontend fejlesztésben, azonban használatuk licenchez kötött. Ezzel szemben a Visual Studio Code ingyenesen elérhető, és megfelelő funkcionalitást nyújt a legtöbb fejlesztési feladathoz.

A verziókezeléshez Git rendszert alkalmaztam, amely lehetővé teszi a kód változásainak nyomon követését és a verziók kezelését. A forráskód tárolása GitHub platformon történt, amely biztosítja a kód biztonságos tárolását és az együttműködés lehetőségét.

A fejlesztés során az API végpontok tesztelésére Postman alkalmazást használtam. Ez az eszköz lehetővé teszi HTTP kérések egyszerű küldését és a válaszok részletes vizsgálatát, ami különösen hasznos a backend fejlesztés során.

A választott fejlesztői környezet és eszközök jól támogatják a modern webalkalmazások fejlesztését, és biztosítják a hatékony, átlátható és jól szervezett munkafolyamatot.

2. fejezet

A rendszer célja és funkcionális követelményei

Az alkalmazásom célja, hogy a vállalati környezetben használt informatikai eszközök (jelenleg számítógépek, laptopok, telefonok, SIM kártyák) kezelésére egy egységes, átlátható és könnyen karbantartható felületet biztosítson, amely támogatja az eszközök teljes életciklusának nyomon követését.

A megvalósítás során kiemelt szempont volt a felhasználóbarát kialakítás és az, hogy a rendszer hosszú távon is rugalmasan bővíthető legyen új eszköztípusokkal és funkciókkal, így alkalmazkodva a változó vállalati igényekhez.

Elvárás továbbá, hogy az alkalmazás bevezetésével jelentősen csökkenjen az adminisztrációs terhek mennyisége, ezáltal hatékonyabbá váljon az eszközök nyilvántartása és több kapacitás szabaduljon fel a mindennapi működés során.

2.1. A rendszer célja

A rendszer alapvető célja, hogy digitalizálja és automatizálja az informatikai eszközök nyilvántartását, valamint megkönnyítse a felhasználók munkáját. Ennek érdekében a fejlesztés során az alábbi célokat határoztam meg:

1. Egységes eszköznyilvántartás létrehozása

   Az összes informatikai eszköz adatai egy központi adatbázisban kerülnek tárolásra. Az eszközről tárolt információ, jelenlegi állapota és felelőse valós időben követhető.

2. Eszközrendelési és jóváhagyási folyamat kialakítása

   A dolgozók az alkalmazáson keresztül adhatják le új eszköz igényüket, amelyet a kiválasztott vezető bírál el. A rendelések feldolgozását az adminisztrátorok végzik. A rendelés feldolgozásához szükséges információkat és eszközöket egy felüle-

ten érnek el. Ez a digitális folyamat helyettesíti a korábbi e-mailes vagy szóbeli engedélyezéseket.

3. Karbantartható és biztonságos rendszer fejlesztése

   A jogosultságkezelés több szintű: a rendszer megkülönbözteti a *felhasználó*, a *vezető* és az *adminisztrátor* szerepköröket.

   Minden felhasználó csak a saját jogosultsági körébe tartozó funkciókat érheti el.

2.2. Funkcionális követelmények

A rendszer funkcionális követelményei meghatározzák azokat a műveleteket és szolgáltatásokat, amelyeket az alkalmazásnak biztosítania kell a felhasználók számára. A követelmények meghatározása során figyelembe vettem a rendszer célját, valamint a különböző felhasználói szerepkörök igényeit.

A rendszer többféle felhasználói szerepkört különböztet meg, például normál felhasználót, vezetőt és adminisztrátort. Az egyes szerepkörök eltérő jogosultságokkal rendelkeznek, amelyek meghatározzák, hogy milyen műveleteket hajthatnak végre az alkalmazásban.

A legfontosabb funkcionális követelmények a következők:

2.2.1. Bejelentkezés és hitelesítés

A rendszer használata kizárólag hitelesített felhasználók számára engedélyezett. Ennek megfelelően az alkalmazás minden funkciója csak bejelentkezést követően érhető el.

A felhasználók a bejelentkezési felületen e-mail címük és jelszavuk megadásával azonosítják magukat. A megadott adatok ellenőrzését a szerver végzi.

Sikeres hitelesítés esetén a rendszer hozzáférést biztosít a felhasználó számára a jogosultsági szintjének megfelelő funkciókhoz. Sikertelen bejelentkezés esetén a rendszer hibaüzenetet jelenít meg, és nem engedélyezi a további műveletek végrehajtását.

A bejelentkezési folyamat biztosítja, hogy az alkalmazás funkciói kizárólag jogosult felhasználók számára legyenek elérhetők.

2.2.2. Felhasználói szerepkör

– Eszközigénylés létrehozása

  A felhasználók a rendelkezésükre álló felületeken keresztül új igényléseket adhatnak le. A számítógépek, laptopok és mobilkészülékek igényléséhez két különálló felület került kialakításra.

  Az igénylés rögzítéséhez az alábbi mezők kitöltése szükséges:

- Eszközkategória:

  A felhasználó legördülő listából választhatja ki a kívánt eszközkategóriát, amelyeket az adminisztrátorok hoznak létre és tartanak karban. A kiválasztott kategória meghatározza az eszköz típusát.

- Ügyféltípus:

  Mobil eszköz igénylése esetén választható mező, amely az adminisztrátorok által definiált értékeket tartalmazza. Az egyes ügyféltípusok eltérő előfizetési csomagokat jelentenek.

- Átvétel helye:

  Az eszköz átvételének helyszíne, amely az elérhető telephelyek közül választható ki.

- Megjegyzés:

  Opcionálisan kitölthető mező, amely lehetőséget biztosít további információk megadására.

- Jóváhagyó:

  A vezetői jogosultsággal rendelkező felhasználók listáját tartalmazó választható mező.

Az igénylés rögzítését követően a felhasználó visszajelzést kap a felületen, továbbá értesítés kerül kiküldésre az igénylő és a kijelölt jóváhagyó részére.

– Igénylések nyomon követése

  A felhasználók a számukra elérhető felületen ellenőrizhetik a korábban leadott igénylések adatait, valamint azok aktuális státuszát.

  A különböző eszköztípusokhoz tartozó igénylések elkülönített felületeken jelennek meg.

  A főoldalon a leadott igénylések táblázatos formában, az alábbi adatokkal kerülnek megjelenítésre:

  - Igénylés azonosító

    Minden igénylés egyedi azonosítóval rendelkezik.

  - Eszköz típusa
  - Átvétel helye
  - Státusz
  - Részletek megtekintése gomb

    Az adott igénylés részletes adatainak megjelenítése.

- Törlés gomb

  Amennyiben a feldolgozás még nem kezdődött meg, az igénylés törölhető.

– Értesítések megtekintése

  A felhasználók a navigációs sáv megfelelő menüpontján keresztül érhetik el értesítéseiket. Az olvasatlan értesítések száma a felületen külön jelölésre kerül.

  Az értesítések listában jelennek meg, ahol az új elemek kiemelten láthatók.

  Értesítés az alábbi eseményekről történik:

  1. Igénylés sikeres rögzítése
  2. Igénylés jóváhagyásra vár
  3. Igénylés elbírálása (jóváhagyás vagy elutasítás)
  4. Feldolgozás megkezdése
  5. Feldolgozás befejezése, eszköz átvehető
  6. Eszköz átadása

– Saját eszközök megjelenítése

  A felhasználók számára elérhető egy eszköztípusonként strukturált felület, ahol saját eszközeik nyilvántartását tekinthetik meg.

  A megjelenítés táblázatos formában történik, az alábbi adatokkal:

  1. Egyedi azonosító
  2. Kategória
  3. Modell
  4. Sorozatszám
  5. Státusz
  6. Státusz indoklása

  A felületen lehetőség van cserefolyamat kezdeményezésére, illetve az eszköz leadásának indítására.

2.2.3. Vezetői szerepkör

A vezetői szerepkörrel rendelkező felhasználók minden felhasználói funkcióval rendelkeznek, továbbá jogosultak a jóváhagyási folyamat kezelésére.

A jóváhagyási funkciók a navigációs sáv megfelelő menüpontján keresztül érhetők el, amely kizárólag ezen szerepkör számára látható.

A vezető kizárólag azokat az igényléseket tekintheti meg, amelyek esetében jóváhagyóként került kijelölésre.

A lista táblázatos formában jelenik meg, és tartalmazza a szükséges adatokat, valamint a részletes nézet elérését biztosító lehetőséget.

A részletes nézetben az igénylés jóváhagyható vagy elutasítható. A döntésről az igénylő értesítést kap. Jóváhagyás esetén az igénylés az adminisztrációs folyamatba kerül.

2.2.4. Adminisztrátori szerepkör

Az adminisztrátori jogosultsággal rendelkező felhasználók a navigációs sáv menüpontjain keresztül érhetik el funkcióikat, amelyek kizárólag számukra elérhetők.

- Felhasználók kezelése

  Felhasználói fiókok létrehozása és karbantartása.

- Eszköznyilvántartás kezelése

  A készleten lévő eszközök rögzítése, valamint adataik és státuszuk naprakészen tartása.

- Eszközkategóriák kezelése

  Az igénylés során választható kategóriák létrehozása, módosítása és törlése.

- Igénylések feldolgozása

  Az adminisztrátor feladata a jóváhagyott igénylések feldolgozása.

  A feldolgozás során az adminisztrátor kiválaszt egy megfelelő, készleten lévő eszközt, majd azt hozzárendeli az adott igényléshez. Mobil eszköz esetén a készülék mellett SIM kártya hozzárendelése is szükséges.

  A hozzárendelést követően az igénylés státusza „Átvehető” értékre módosul, és a felhasználó értesítést kap.

  Az eszköz átvételét követően az adminisztrátor lezárja az igénylést, amelynek státusza „Kész” értékre változik. Ettől a ponttól kezdve az eszközért a felhasználó felel.

2.2.5. Nem funkcionális követelmények

A rendszer biztonságának és könnyen karbantarthatóságának érdekében az alábbi követelményeket határoztam meg:

- Biztonság: A kommunikáció JWT token alapú hitelesítéssel történik, ezzel biztosítva az adatokhoz való hozzáférés korlátozását.

- Teljesítmény: A .NET 8 és a React technológiák révén az alkalmazás gyors válaszidőt biztosít még nagyobb adatbázis esetén is.
- Reszponzivitás: A frontend mobilbarát kialakítású, így bármilyen eszközről kényelmesen használható.
- Karbantarthatóság: A komponensalapú React frontend és a rétegzett .NET architektúra biztosítja a könnyű bővíthetőséget.
- Megbízhatóság: A fejlesztés során az API végpontokat tesztelésekkel és hibakezeléssel is megerősítettem.

2.2.6. Hasonló rendszerek bemutatása

Az általam fejlesztett rendszer funkcionalitása több, már létező megoldással is párhuzamba állítható. Ilyen például a Snipe-IT, amely szintén informatikai eszközök nyilvántartására és kezelésére szolgál. Hasonló funkcionalitást kínál a GLPI rendszer is, amely azonban egy komplexebb, IT szolgáltatásmenedzsment megoldás. Emellett megemlíthető a ServiceNow platform, amely vállalati környezetben biztosít fejlett workflow és jóváhagyási lehetőségeket. A fejlesztett alkalmazás ezekhez képest egy célzottabb, egyszerűbb megoldást kínál, amely kifejezetten az eszközkezelési folyamatokra koncentrál.

2.2.7. Összegzés

A fejlesztett alkalmazás célja, hogy egy modern, biztonságos és felhasználóbarát rendszer segítségével támogassa a vállalat informatikai eszközeinek kezelését. Az egységesített folyamatok és az egyértelmű szerepkörök révén a rendszer csökkenti az adminisztrációs terheket, miközben növeli az átláthatóságot és a hatékonyságot a szervezeten belül.

3. fejezet

Tervezés

Az alábbi fejezetben bemutatom a rendszer tervezése során meghozott döntéseket és az alkalmazott technológiák részleteit. Bemutatásra kerül a háromrétegű architektúra, az adatbázis tervezése és normalizálása, az üzleti logikai felépítésének kialakítása és a kliens oldali legfőbb komponensek.

3.1. Rendszer architektúra

A fejlesztett alkalmazás háromrétegű architektúrát követ, amely jól elkülöníti a megjelenítési, az üzleti logikai és az adatkezelési rétegeket.

A modell legnagyobb előnye, hogy lehetővé teszi az egyes rétegek egymástól függetlenül történő fejlesztését, sőt, akár teljes cseréjét is, lépést tartva így a folyamatosan változó követelményekkel és az egyre újabb technológiákkal. Ez biztonságosan megtehető, mert egy réteg módosítása nincs hatással a többi réteg működésére. Egymástól független modulokként tartalmazza a felhasználói felületet, az üzleti logikát és az adatbázist a szükséges hozzáférési műveletekkel. [6]

A háromrétegű architektúra fő komponensei a következők:

- Frontend (kliensoldal): A felhasználói felület React keretrendszer használatával készült. A frontend feladata az interakciók kezelése, az adatok megjelenítése, valamint az API hívások indítása a backend felé.

- Backend (szerveroldal): A szerveroldali logikát .NET 8 Web API biztosítja. Ez a réteg kezeli az üzleti logikát, feldolgozza a kliens kéréseit, valamint kommunikál az adatbázissal.

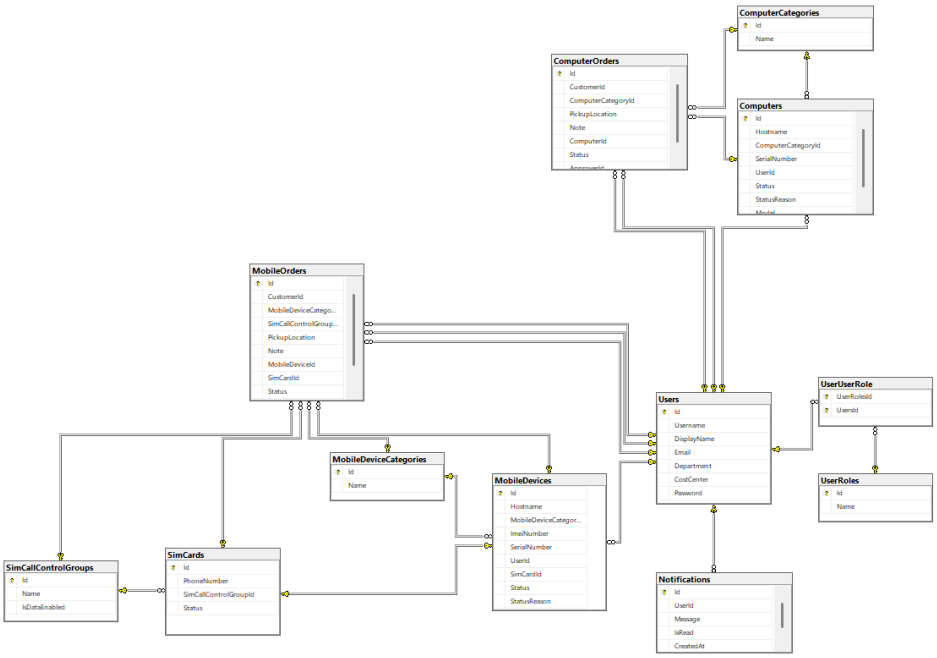
- Adatbázis: Az adatok tárolása Microsoft SQL Server adatbázisban történik, amely strukturált és megbízható adattárolást biztosít.

– A frontend és a backend közötti kommunikáció REST API-n keresztül valósul meg HTTP protokoll segítségével. Az adatcsere JSON formátumban történik, amely könnyen feldolgozható mindkét oldalon.

Az architektúra egyik fontos előnye, hogy a frontend és a backend egymástól függetlenül fejleszthető és skálázható. Ez lehetővé teszi a rendszer későbbi bővítését vagy akár más kliensalkalmazások (pl. mobilalkalmazás) integrálását is.

3.2. Adatbázis tervezés

A rendszer működésének alapját egy relációs adatbázis képezi, amely biztosítja az adatok strukturált és konzisztens tárolását. Az adatbázis tervezése során a cél az volt, hogy az eszközök, felhasználók és rendelések közötti kapcsolatok egyértelműen és hatékonyan reprezentálhatók legyenek.



3.1. ábra. Adatbázis terv

A táblák között az alábbi relációk vannak kialakítva:

Egy felhasználó több rendelést is létrehozhat. Egy eszköz egy adott időpontban egy felhasználóhoz tartozhat. Egy felhasználó több eszközzel is rendelkezhet. A felhasználók több szerepkörrel rendelkezhetnek.

Az adatbázis tervezése során fontos szempont volt a normalizáció, amely csökkenti az adatredundanciát és biztosítja az adatok integritását.

3.3. Backend tervezése

A backend réteg a .NET 8 Web API keretrendszerre épül, és rétegzett architektúrát követ.

A rendszer főbb komponensei:

- Kontroller réteg: A HTTP kérések fogadásáért és a válaszok visszaküldéséért felelős. Minden erőforráshoz külön kontroller tartozik.
- Szolgáltatás réteg: Az üzleti logika itt kerül megvalósításra. Ez a réteg végzi az adatok feldolgozását, ellenőrzését és a műveletek végrehajtását.
- Adatkezelési réteg: Az adatbázissal való kommunikációért felel. A Repository pattern alkalmazásával az adatkezelési logika elkülönül az üzleti logikától, így a rendszer könnyebben tesztelhető és karbantartható.

Az alkalmazás az Entity Framework Core ORM-et használja, amely lehetővé teszi az objektum-orientált adatkezelést és az SQL lekérdezések absztrakcióját.

A backend kialakítása során fontos szempont volt a tiszta kód (clean code) elvek követése és a jól strukturált, moduláris felépítés.

3.4. Frontend tervezése

A frontend alkalmazás React keretrendszer használatával, komponensalapú architektúrát követve készült el. Ez lehetővé teszi az alkalmazás logikai egységekre bontását és az újrahasznosítható komponensek használatát.

A főbb felületi elemek:

- Bejelentkezési oldal
- Főoldal
- Eszközlista
- Rendelési felületek
- Adminisztrációs felületek

A navigációt React Router biztosítja, amely lehetővé teszi az egyoldalas alkalmazás (SPA) működést.

Az állapotkezelés a React beépített hook-jaival (pl. useState, useEffect) történik. Az API kommunikáció Axios könyvtár segítségével valósul meg.

A felhasználói felület kialakításakor fontos szempont volt a reszponzív megjelenés, amelyet Tailwind CSS keretrendszer segítségével valósítottam meg.

3.5. Autentikáció és jogosultságkezelés

A rendszer biztonságos működésének érdekében JWT (JSON Web Token) alapú autentikáció került bevezetésre.

A bejelentkezés folyamata a következő:

A felhasználó megadja az e-mail címét és jelszavát. A backend ellenőrzi az adatokat. Sikeres hitelesítés esetén a szerver egy JWT tokent generál. A kliens ezt a tokent eltárolja, és minden további kérésnél elküldi.

A jogosultságkezelés szerepkör alapú (role-based access control). A rendszer különböző felhasználói szerepköröket különböztet meg:

- Felhasználó
- Vezető
- Adminisztrátor

Minden szerepkör eltérő jogosultságokkal rendelkezik, amelyeket a backend oldalon ellenőriz a rendszer. Ez biztosítja, hogy a felhasználók csak a számukra engedélyezett műveleteket hajthassák végre.

A biztonság növelése érdekében a jelszavak titkosított (hash-elt) formában kerülnek tárolásra, és az API végpontok védettek jogosultság ellenőrzéssel.

Az alábbi kódrészlet a felhasználói bejelentkezés folyamatát mutatja be az üzleti logikai oldalról. A kliens HTTP kérésen keresztül JSON formátumban küldi el a kérést a backend részére. A kérésben található a felhasználó által megadott e-mail cím és jelszó ami a bejelentkezéshez szükséges. A program ellenőrzi, hogy a megadott e-mail cím megtalálható-e az adatbázisban, majd alkalmazva a felhasználó létrehozáskor is használt hash algoritmust ellenőrzi a megadott jelszót is. Amennyiben az egyik beküldött adat nem megfelelő a kérés elutasításra kerül. Ha helyes e-mail cím jelszó páros került megadásra, akkor a szolgáltatás egy JSON Token-t ad vissza a válaszüzenetben amit a kliens dolgoz fel.

3.1. kód. Bejelentkezés, hitelesítés

```
1  [HttpPost("/login")]
2  [AllowAnonymous]
3  public async Task<IActionResult> Login(LoginRequest request)
4  {
5      try {
6          var user = await _context.Users
7            .Include(x => x.UserRoles)
8            .FirstOrDefaultAsync(u => u.Email == request.Email);
9
10         if (user == null)
11             return Unauthorized();
12
13         var passwordHasher = new PasswordHasher<User>();
14         var result = passwordHasher.VerifyHashedPassword(
15           user, user.Password, request.Password);
16
17         if (result == PasswordVerificationResult.Failed)
18             return Unauthorized();
19
20         var newTokenRequest = user.ToNewTokenRequest();
21
22         var access_token = _tokenGenerator.GenerateToken(
              newTokenRequest);
23
24         return Ok(access_token);
25     }
26     catch (Exception e) {
27         return BadRequest(e.Message);
28     }
29 }
```

3.6. JSON Web Token működése

A JSON Web Token (JWT) egy nyílt szabvány (RFC 7519), amely kompakt, önmagát tartalmazó módot biztosít a felek közötti biztonságos információcserére JSON objektumként. Gyakran használják modern, állapot nélküli (stateless) webes alkalmazásokban hitelesítésre (authentication) és engedélyezésre (authorization), mivel a token magában hordozza a felhasználói adatokat, aláírása révén pedig hamisíthatatlan. [1]

A token felépítése:

- Header

  A header rész általában két elemet tartalmaz, a token típusát és az alkalmazott aláírási algoritmust például HMAC SHA256 vagy RSA.

- Payload

  A token második része a payload, amely az úgynevezett állításokat (claims) tartalmazza. Az állítások egy adott entitásra – jellemzően a felhasználóra – vonatkozó információkat, valamint további adatokat foglalnak magukba. Az állítások három fő típusa különböztethető meg: regisztrált (registered), nyilvános (public) és privát (private) állítások.

- Signature

  Az aláírás létrehozásához a titkosított fejlécet, a titkosított payloadot, egy titkos kulcsot, valamint a fejlécben megadott algoritmust kell felhasználni, majd ezek alapján történik az aláírás generálása.

4. fejezet

Fejlesztés és megvalósítás

A következő fejezetben bemutatom a fejlesztés során megvalósított megoldásokat. Kódrészleteket mutatok be illetve azokat az intézkedéseket, amiket a biztonság és nyomon követhetőség miatt vezettem be. Bemutatásra kerül az alkalmazott fejlesztői környezet és a komponensek közötti integráció.

4.1. Fejlesztési környezet

A fejlesztés során modern és széles körben használt eszközöket alkalmaztam a hatékony munkavégzés érdekében. A backend fejlesztése Microsoft Visual Studio 2022 környezetben történt, amely teljes körű támogatást nyújt a .NET alapú alkalmazások fejlesztéséhez. A frontend fejlesztés Visual Studio Code szerkesztőben zajlott, amely könnyű, gyors és kiválóan alkalmas JavaScript alapú projektek kezelésére.

A verziókezeléshez Git rendszert használtam, amely lehetővé tette a fejlesztési folyamat követését, valamint a kód biztonságos tárolását. A forráskód GitHub platformon került tárolásra.

Az API végpontok teszteléséhez Postmant használtam.

4.2. Backend fejlesztés

A backend megvalósítása .NET 8 Web API keretrendszer segítségével történt. Az alkalmazás REST alapú szolgáltatásokat biztosít, amelyek HTTP kéréseken keresztül érhetők el.

A rendszer több különálló vezérlőt (controller) tartalmaz, amelyek az egyes erőforrások kezeléséért felelősek. Ilyenek például:

- DeviceController – eszközök kezelése
- OrderController – rendelések kezelése

– AuthController – felhasználói hitelesítés

A backend működésének alapja a rétegzett architektúra, amelyben a controller réteg a kérések fogadásáért, míg a repository réteg az adatbázis műveletekért felel.

Egy tipikus működési folyamat a következő:

1. A kliens HTTP kérést küld egy adott végpontra.
2. A controller fogadja a kérést és továbbítja a megfelelő szolgáltatásnak.
3. A repository réteg lekérdezi vagy módosítja az adatokat az adatbázisban.
4. A backend JSON formátumban visszaküldi a választ a kliensnek.

Az alábbi kódrészlet tartalmazza a számítógép rendelés létrehozását az üzleti logika oldalról. A felhasználó HTTP kérést küld a backend felé, ami JSON formátumban tartalmazza a megrendelés létrehozásához szükséges adatokat. A beérkező kérésből egy adatbázis modellt készítek, ami a következő rétegben kerül mentésre az adatbázisba. A folyamatban szerepel a felhasználói értesítés létrehozása is. Amennyiben az adatbázisba való adatmentés sikeres, a visszatérési modellt átalakítom a kliens számára szükséges formátumra, amit a visszatérési értékben adok tovább.

4.1. kód. Számítógép rendelés létrehozása végpont

```
1  [HttpPost("/computer-orders")]
2  [Authorize(Roles = "User, Group leader")]
3  public async Task<ActionResult<ComputerOrderResponse>> Create([
      FromBody] CreateComputerOrderRequest request)
4  {
5    try {
6      var order = await ComputerOrder.Create(
7      request,
8      _userRepository,
9      _computerCategoryRepository);
10
11     await _computerOrderRepository.CreateAsync(order);
12
13     await _notificationService.SendAsync(
14     order.ApproverId,
15     $"New computer order from {order.Customer.DisplayName} is
          waiting for your approval.");
16
17     var response = order.ToResponse();
18
```

```
19      return CreatedAtAction(nameof(GetById), new { id = response
            .Id }, response);
20    }
21    catch (Exception e) {
22      return BadRequest(e.Message);
23    }
24  }
```

Az adatkezeléshez Entity Framework Core keretrendszert használtam, amely lehetővé teszi az objektum-orientált adatbázis-kezelést. A lekérdezések LINQ segítségével valósulnak meg. A biztonság érdekében a felhasználók hitelesítése JWT tokenekkel történik, és az API végpontok védettek jogosultság ellenőrzéssel.

Az alábbi kódrészletben bemutatom a számítógép rendelés létrehozását a repository réteg szintjén. A beérkező adatokat validációkkal láttam el, amik ellenőrzik a beküldött adatok érvényességét.

4.2. kód. Számítógép rendelés model létrehozása

```
1  public static async Task<ComputerOrder> Create(
2      CreateComputerOrderRequest request,
3      IUserRepository userRepository,
4      IComputerCategoryRepository computerCategoryRepository
5  )
6  {
7      var customer = await userRepository.GetByIdAsync(request.
           CustomerId);
8
9      if (customer == null)
10         throw new Exception("Customer is not found!");
11
12     var category = await computerCategoryRepository.
           GetByIdAsync(request.ComputerCategoryId);
13
14     if (category == null)
15         throw new Exception("Computer category not found!");
16
17     var approver = await userRepository.GetByIdAsync(request.
           ApproverId);
18
19     if (approver == null)
20         throw new Exception("Approver is not found!");
21
22     return new ComputerOrder()
```

```
23      {
24          CustomerId = customer.Id,
25          Customer = customer,
26          ComputerCategoryId = category.Id,
27          ComputerCategory = category,
28          PickupLocation = request.PickupLocation,
29          Note = request.Note,
30          Status = "Waiting for approval",
31          ApproverId = approver.Id,
32          Approver = approver
33      };
34  }
```

Amennyiben az adatok validációja sikeres és nincs eltérés akkor létrehozom az adatbázis modellt és az Entity Framework Core használatával mentem az adatbázisba.

4.3. kód. Számítógép rendelés mentése az adatbázisba

```
1  public async Task<ComputerOrder> CreateAsync(ComputerOrder
       order)
2  {
3      await _context.ComputerOrders.AddAsync(order);
4
5      await _context.SaveChangesAsync();
6
7      return order;
8  }
```

4.3. Frontend fejlesztés

A frontend fejlesztése React keretrendszer segítségével történt. Az alkalmazás egy egyoldalas alkalmazásként (Single Page Application, SPA) működik, amely dinamikusan tölti be az adatokat a backend API-ból.

A felület komponensalapú felépítésű, amely lehetővé teszi az egyes elemek újrafelhasználását és egyszerű karbantartását. A főbb komponensek közé tartoznak:

- bejelentkezési felület
- eszközlista megjelenítő komponens
- rendelési űrlap
- adminisztrációs felület

A navigációt React Router biztosítja, amely lehetővé teszi az oldalak közötti gyors váltást újratöltés nélkül.

Az API hívások Axios könyvtáron keresztül történnek. A frontend a backend által biztosított REST végpontokat használja az adatok lekérésére és módosítására.

Az állapotkezelés a React hook-jain keresztül valósul meg, például useState és useEffect használatával. Ezek segítségével a felhasználói interakciók és az adatok változásai hatékonyan kezelhetők.

A felhasználói felület kialakításakor fontos szempont volt az egyszerű kezelhetőség és az átlátható megjelenés, amelyet Tailwind CSS segítségével valósítottam meg.

Az alábbi kódrészleten keresztül bemutatom a React könyvtár által biztosított komponensalapú fejlesztés lehetőségét. A kódrészlet egy többször felhasznált gombot generáló kódrészletet tartalmaz, ami a paraméterlistájának köszönhetően dinamikusan alkalmazható.

4.4. kód. Komponensalapú fejlesztés példa

```
1  export default function Button({
2    handleClick,
3    color,
4    label,
5  }: {
6    handleClick: () => void;
7    color: "red" | "green" | "gray" | "blue" | "yellow";
8    label: string;
9  }) {
10   let classes = "text-white px-2 py-1 rounded transition cursor
       -pointer";
11
12   if (color == "red") classes += " bg-red-600 hover:bg-red-800"
       ;
13   else if (color == "gray") classes += " bg-gray-300 hover:bg-
       gray-800";
14   else if (color == "blue") classes += " bg-blue-600 hover:bg-
       blue-800";
15   else if (color == "yellow") classes += " bg-yellow-500 hover:
       bg-yellow-800";
16   else if (color == "green") classes += " bg-green-500 hover:bg
       -green-800";
17
18   return (
19     <button onClick={handleClick} className={classes}>
20       {label}
```

```
21      </button>
22    );
23  }
```

4.4. Integráció

A frontend és a backend közötti integráció REST API-n keresztül valósul meg. A kliensoldali alkalmazás HTTP kéréseket küld a backend végpontjai felé, amelyek feldolgozzák a kéréseket és visszaküldik a szükséges adatokat.

Az autentikáció során a frontend eltárolja a bejelentkezéskor kapott JWT tokent, majd minden további API hívás során elküldi azt az Authorization fejlécben.

A rendszer biztosítja a hibakezelést is, amennyiben egy API hívás sikertelen, a frontend megfelelő visszajelzést ad a felhasználónak.

Az integráció során külön figyelmet fordítottam arra, hogy a frontend és backend közötti adatcsere egységes és jól strukturált legyen.

4.5. Biztonság és naplózás

A rendszer biztonságos működése érdekében több védelmi mechanizmust alkalmaztam.

A felhasználók hitelesítése JWT token alapú megoldással történik, amely biztosítja, hogy csak jogosult felhasználók férhessenek hozzá az API végpontokhoz.

A jelszavak nem kerülnek tárolásra nyílt formában, hanem titkosított (hash-elt) módon kerülnek az adatbázisba. Ez jelentősen növeli az adatok biztonságát.

A rendszer tartalmaz alapvető hibakezelést is, amely biztosítja, hogy a váratlan hibák ne okozzanak rendszerleállást.

A naplózás (logging) lehetővé teszi a rendszer működésének nyomon követését, valamint a hibák későbbi elemzését. A naplózott események közé tartozhatnak például a bejelentkezések, hibák és fontosabb műveletek.

5. fejezet

Tesztelés

Az alábbi fejezetben bemutatom az alkalmazott tesztelési módszertanokat és azok eredményeit. A tesztelés során manuális tesztelést végeztem és az API végpontok működését Postman alkalmazással ellenőriztem.

5.1. Tesztelési módszertan

A fejlesztett rendszer megfelelő működésének biztosítása érdekében különböző tesztelési módszereket alkalmaztam. A tesztelés célja az volt, hogy ellenőrizzem az alkalmazás funkcionalitását, megbízhatóságát és stabilitását.

A tesztelés során elsősorban manuális tesztelést végeztem, amely során a rendszer különböző funkcióit valós felhasználói szemszögből próbáltam ki. Emellett az API végpontok működését külön is ellenőriztem Postman segítségével.

Az alábbi tesztelési típusokat alkalmaztam:

- Funkcionális tesztelés: Annak ellenőrzése, hogy az egyes funkciók (pl. bejelentkezés, eszköz rendelés, jóváhagyás) megfelelően működnek-e.
- Integrációs tesztelés: A frontend és a backend közötti kommunikáció vizsgálata, különös tekintettel az API hívások helyes működésére.
- Hibakezelés tesztelése: Annak ellenőrzése, hogy a rendszer hogyan reagál hibás bemenetekre vagy sikertelen műveletekre.
- Jogosultság tesztelése: A különböző szerepkörök (felhasználó, vezető, adminisztrátor) jogosultságainak ellenőrzése.

A tesztelés során törekedtem arra, hogy minden fontos funkciót több különböző esettel is lefedjek.

5.2. Tesztesetek és eredmények

A rendszer működésének ellenőrzésére több tesztesetet definiáltam, amelyek lefedik az alkalmazás fő funkcióit.

Teszteset	Leírás	Várt eredmény	Eredmény
T1	Helyes e-mail és jelszó megadása	Sikeres bejelentkezés, JWT token visszaadása	Megfelelt
T2	Hibás jelszó megadása	Hibaüzenet jelenik meg	Megfelelt
T3	Nem létező felhasználó megadása	Hibaüzenet jelenik meg	Megfelelt

5.1. táblázat. Bejelentkezési funkció tesztelése

Teszteset	Leírás	Várt eredmény	Eredmény
T4	Új eszközrendelés létrehozása érvényes adatokkal	A rendelés mentésre kerül az adatbázisban	Megfelelt
T5	Rendelés létrehozása hiányos adatokkal	Hibaüzenet jelenik meg	Megfelelt
T6	Saját rendelések lekérdezése	A felhasználó rendelései helyesen megjelennek	Megfelelt

5.2. táblázat. Eszközrendelési funkció tesztelése

Teszteset	Leírás	Várt eredmény	Eredmény
T7	Vezető jóváhagy egy rendelést	A rendelés státusza „jóváhagyva” lesz	Megfelelt
T8	Vezető elutasít egy rendelést	A rendelés státusza „elutasítva” lesz	Megfelelt

5.3. táblázat. Jóváhagyási folyamat tesztelése

Teszteset	Leírás	Várt eredmény	Eredmény
T9	Normál felhasználó admin felület elérése	Hozzáférés megtagadva	Megfelelt
T10	Adminisztrátor új eszközt hoz létre	Az eszköz sikeresen mentésre kerül	Megfelelt

5.4. táblázat. Jogosultságkezelés tesztelése

5.3. API tesztelés (Postman)

Az API végpontok külön tesztelésre kerültek Postman segítségével. Az ellenőrzések során az alábbiakat vizsgáltam:

- helyes HTTP státuszkódok (200, 400, 401, 404)
- megfelelő JSON válaszformátum
- autentikáció működése (token szükségessége)

A tesztek alapján az API végpontok megbízhatóan működnek, és megfelelően kezelik a hibás kéréseket is.

5.4. Összegzés

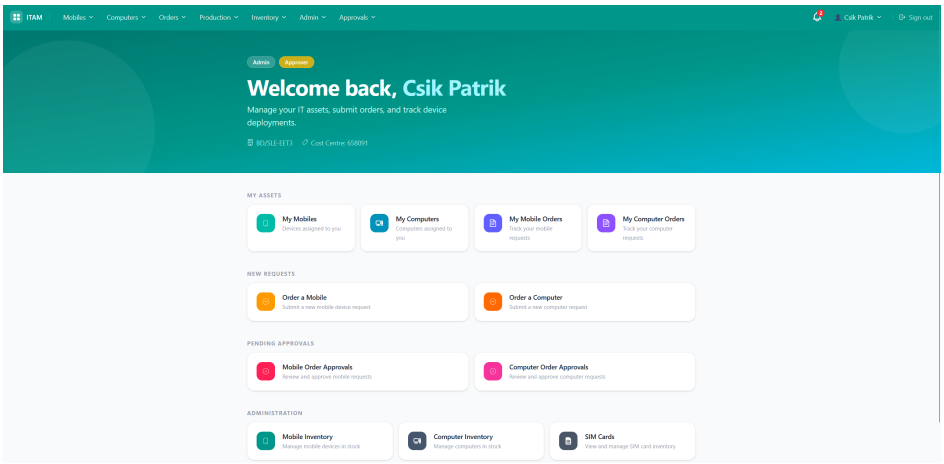
A tesztelési folyamat során a rendszer minden fő funkcióját sikeresen ellenőriztem. A tesztek eredményei alapján az alkalmazás stabilan működik, és megfelel a korábban meghatározott követelményeknek.

A feltárt kisebb hibák javításra kerültek a fejlesztési folyamat során, így a végleges rendszer egy megbízható és jól használható megoldást nyújt az informatikai eszközök kezelésére.

6. fejezet

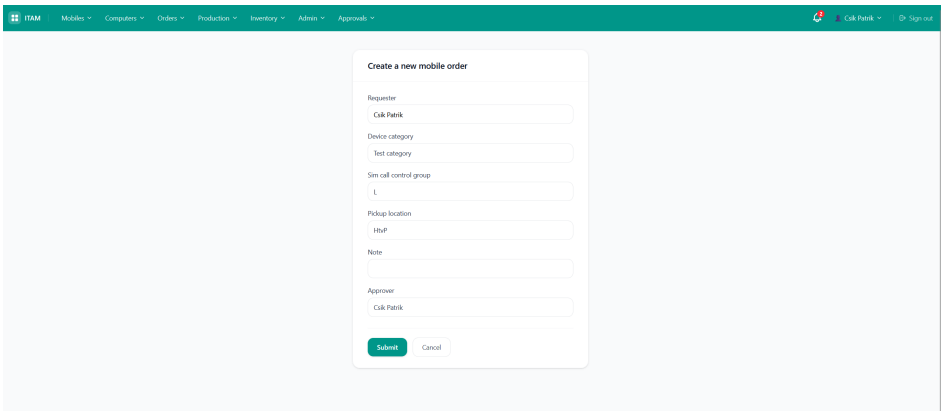
Képernyőképek és felhasználói felületek bemutatása

A 6.1. ábrán látható az alkalmazás kezdőképernyője felhasználói bejelentkezés után. A felületen elérhetők az rendszer fő funkciói és azoknak rövid leírása. Megjelenítésre kerül a bejelentkezett felhasználó legfontosabb adatai is.



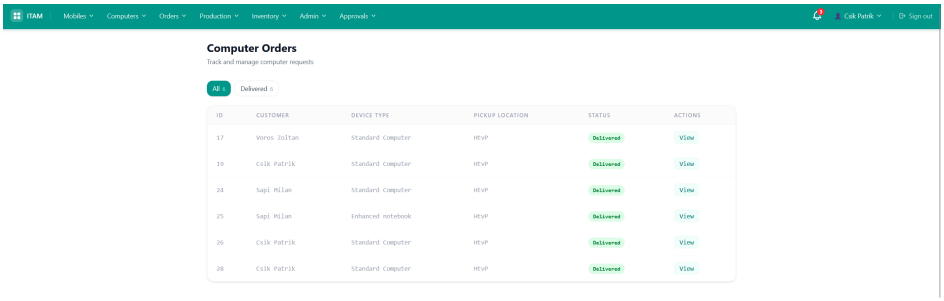
6.1. ábra. Kezdőlap

A 6.2. ábrán látható a mobil készülék rendelés folyamatának elindítása. Az űrlapon alapértelmezetten kitöltésre kerül az igénylő személy a bejelentkezési adatok alapján. A megrendelő kiválaszthatja a számára szükséges készülék kategóriát, az igényelt SIM kártya ügyféltípusát, az átvétel helyét, a jóváhagyót és opcionálisan megjegyzést tehet a megrendelésével kapcsolatban.



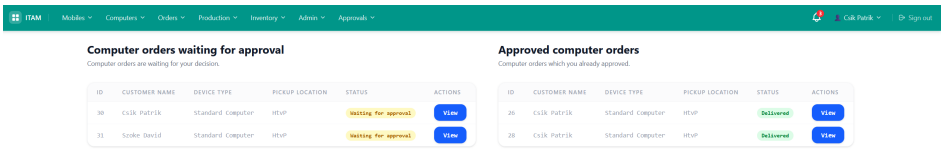
6.2. ábra. Mobil készülék igénylése

A 6.3. ábrán az adminisztrátorok által elérhető számítógép rendelések nyomon követési felület látható. A megrendelések időrendi sorrendben jelennek meg a táblázatban. A felületet elérő kollégák megtekinthetik a megrendeléshez tartozó legfontosabb adatokat mint az igénylő nevét, az igényelt eszköz kategóriáját és a rendelés jelenlegi státuszát. A megrendelés további részleteit a View gombbon keresztül érhetik el.



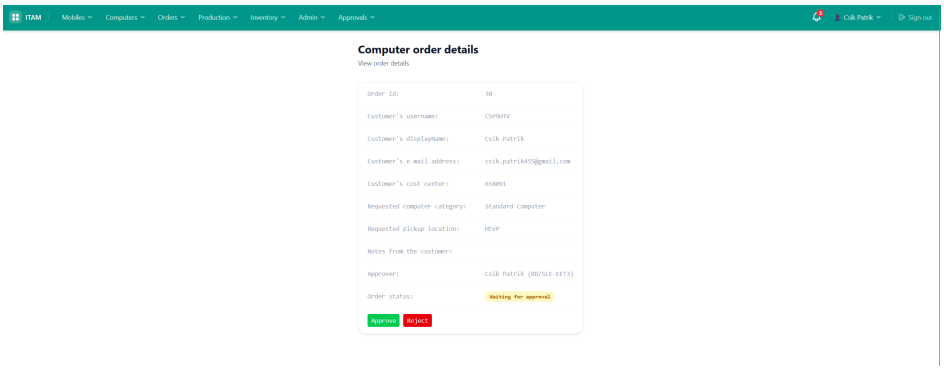
6.3. ábra. Számítógép rendelések

A 6.4. ábrán látható menüpont a vezetői jogosultsággal rendelkező felhasználók számára érhető el és az általuk jóváhagyásra váró, vagy már korábban jóváhagyott számítógép rendeléseket és azok jelenlegi státuszát tekinthetik meg.



6.4. ábra. Jóváhagyásra váró számítógép rendelések

A 6.5. ábrán a számítógép rendelés jóváhagyása látható. A Megjelenített adatok alapján, a megadott vezető elbírálhatja a számítógép rendelést.



6.5. ábra. Számítógép rendelés jóváhagyása

7. fejezet

További fejlesztési lehetőségek

A fejlesztett alkalmazás jelenlegi formájában jól használható megoldást nyújt az informatikai eszközök nyilvántartására és kezelésére, azonban számos olyan fejlesztési lehetőség megvalósítható, amelyek tovább növelhetik a rendszer hatékonyságát és felhasználói élményt.

Az egyik legfontosabb jövőbeli fejlesztési irány az e-mail alapú értesítési rendszer bevezetése. Ennek segítségével a felhasználók automatikus értesítést kaphatnának például új eszközrendelés létrehozásakor, annak jóváhagyásáról vagy elutasításáról, illetve eszköz visszavétel esetén. Ez jelentősen javítaná a kommunikáció hatékonyságát, és csökkentené annak esélyét, hogy egy-egy fontos esemény elkerülje a felhasználók figyelmét.

A rendszer funkcionalitása bővíthető lenne egy statisztikai és riportáló modul bevezetésével is. Ennek segítségével a vezetők különböző kimutatásokat készíthetnének, például az eszközök kihasználtságáról, a rendelések számáról vagy az egyes eszköztípusok eloszlásáról. Ezek az adatok támogatnák a stratégiai döntéshozatalt.

Egy másik lehetséges fejlesztés a mobilalkalmazás létrehozása, amely lehetővé tenné a rendszer használatát okostelefonról is. Ez különösen hasznos lehet olyan környezetben, ahol a felhasználók gyakran dolgoznak irodán kívül.

A biztonság tovább növelhető lenne kétfaktoros hitelesítés (2FA) bevezetésével, amely további védelmi réteget biztosít a felhasználói fiókok számára.

Végül a rendszer bővíthető lenne külső rendszerekkel való integrációval, például vállalati címtárakkal (pl. Active Directory), amely lehetővé tenné az automatikus felhasználókezelést és az egységes hitelesítést.

Összességében elmondható, hogy a fejlesztett alkalmazás jó alapot biztosít további funkciók bevezetéséhez, és megfelelő továbbfejlesztéssel egy komplex, vállalati szintű eszközkezelő rendszerré alakítható.

8. fejezet

Összefoglalás és köszönetnyilvánítás

A szakdolgozat célja egy olyan webalapú alkalmazás megtervezése és megvalósítása volt, amely támogatja az informatikai eszközök nyilvántartását, valamint az eszközrendelési és jóváhagyási folyamatok kezelését. A dolgozat során egy olyan rendszer került kialakításra, amely képes a vállalati környezetben gyakran előforduló adminisztrációs problémák hatékony kezelésére.

A fejlesztés során modern technológiákat alkalmaztam mind a backend, mind a frontend oldalon. A backend megvalósítása .NET 8 Web API keretrendszerrel történt, míg a felhasználói felület React alapokon került kialakításra. Az adatkezelést relációs adatbázis biztosítja, amely lehetővé teszi az adatok közötti kapcsolatok strukturált és megbízható kezelését.

A dolgozat részletesen bemutatta a rendszer tervezését, beleértve az architektúra kialakítását és az adatbázis megtervezését, valamint ismertette a megvalósítás során alkalmazott technológiai megoldásokat. A tesztelési fejezet igazolta, hogy az alkalmazás stabilan működik és megfelel a meghatározott funkcionális követelményeknek.

A fejlesztés során szerzett tapasztalatok hozzájárultak a modern webalkalmazások fejlesztéséhez szükséges ismereteim elmélyítéséhez, különös tekintettel a kliens–szerver architektúrára, a REST API-k kialakítására, valamint a jogosultságkezelés megvalósítására.

Összességében elmondható, hogy a kitűzött célokat sikerült megvalósítani, és egy működőképes, jól bővíthető rendszert hoztam létre. A dolgozatban bemutatott megoldás megfelelő alapot biztosít további fejlesztésekhez, és a jövőben akár komplexebb vállalati környezetben is alkalmazható.

Ezúton szeretném megköszönni témavezetőmnek Dr. Király Rolandnak a dolgozat elkészítése során nyújtott segítségét, szakmai iránymutatását és támogatását, amely nagyban hozzájárult a szakdolgozat sikeres megvalósításához.

9. fejezet

Forráskód és demó alkalmazás elérhetősége

Az elkészített alkalmazás teljes forráskódja elérhető a GitHub oldalamon. Elérhetősége: https://github.com/csik-patrik/Thesis

A demó alkalmazás publikálásához vásároltam egy szolgáltató által biztosított Ubuntu szervert, amit webszerverként üzemeltetek. Az alkalmazás élesítéséhez létrehoztam egy docker containert. A docker konfiguráció elkészítéséhez a Claude Sonnet 4.6-os LLM modelt használtam.

A demó alkalmazás elérhetősége: http://91.236.195.159:3000/

Irodalom

[1] JWT.IO. *JSON Web Token bevezetés*. URL: https://www.jwt.io/introduction#what-is-json-web-token.

[2] Microsoft. *Bevezetés a .NET használatába*. URL: https://learn.microsoft.com/hu-hu/dotnet/core/introduction?WT.mc_id=dotnet-35129-website.

[3] Microsoft. *Entity Framework Core*. URL: https://learn.microsoft.com/hu-hu/ef/core/.

[4] Stack Overflow. *Legnépszerűbb webfejleszési technológiák*. URL: https://survey.stackoverflow.co/2025/technology#most-popular-technologies-language-language-prof.

[5] React. *React*. URL: https://react.dev/.

[6] Wikipedia. *Háromrétegű architektúra*. URL: https://hu.wikipedia.org/wiki/T%C3%B6bbr%C3%A9teg%C5%B1_architekt%C3%BAra.

[7] Wikipedia. *Microsoft SQL Server*. URL: https://en.wikipedia.org/wiki/Microsoft_SQL_Server.

Nyilatkozat

Alulírott *Csik Patrik*, büntetőjogi felelősségem tudatában kijelentem, hogy az általam benyújtott, *Informatikai eszközök nyilvántartására, eszközkiadás kezelésére tervezett alkalmazás készítése* című szakdolgozat önálló szellemi termékem. Amennyiben mások munkáját felhasználtam, azokra megfelelően hivatkozom, beleértve a nyomtatott és az internetes forrásokat is.

Aláírásommal igazolom, hogy az elektronikusan feltöltött és a papíralapú szakdolgozatom formai és tartalmi szempontból mindenben megegyezik.

Eger, 2026. április 11.

Csik Patrik

aláírás